

HomeAssist AI

****HomeAssist AI**** is a smart household service marketplace platform that connects users with nearby workers for cleaning, garbage collection, junk removal, shifting assistance, and other household tasks.

🚀 Tech Stack & Libraries Used

Backend: Django & Django REST Framework

The backend is a robust Python-based server that handles API requests, user authentication, and database management.

- ****Django (6.0.5)****: Core web framework.
- ****Django REST Framework (3.17.1)****: Used to build powerful and scalable RESTful APIs.
- ****djanoestframework-simplejwt (5.5.1)****: Handles JSON Web Token (JWT) authentication for secure, stateless user sessions.
- ****Django Cors Headers****: Enables Cross-Origin Resource Sharing so the React Native app can communicate with the backend.
- ****Django Channels & Daphne (4.3.2 / 4.2.1)****: ASGI setup required for WebSockets, enabling real-time worker tracking and live notifications.
- ****Pillow (12.2.0)****: Python Imaging Library, necessary for handling `ImageField` uploads (e.g., service icons, before/after work verification images).

Frontend: React Native (Expo)

The frontend is a cross-platform mobile application providing a highly professional and responsive user interface.

- ****React Native & Expo****: Framework for building native iOS and Android apps using React.
- ****@react-navigation/native & @react-navigation/bottom-tabs & @react-navigation/native-stack****: Handles navigation between screens and role-based tab routing (Customer vs. Worker).
- ****Zustand****: A small, fast, and scalable bearbones state-management solution used for managing User Sessions and the Service Cart.
- ****Axios****: Promise-based HTTP client used to interact with the Django REST APIs securely with JWT interceptors.
- ****react-native-vector-icons (Ionicons)****: Standard, beautiful vector icons used throughout the UI.

🛠️ How to Run the Project Locally

1. Starting the Backend (Django)

1. Open a terminal and navigate to the project root directory.

2. Activate the Python virtual environment:

- **Windows**: `.\venv\Scripts\activate``
- **Mac/Linux**: ``source venv/bin/activate``

3. Navigate into the backend folder:

```
``bash
cd backend
...`
```

4. Run database migrations (only needed once or after model changes):

```
``bash
python manage.py makemigrations
python manage.py migrate
...`
```

5. Create a superuser (optional, to access the Django admin panel):

```
``bash
python manage.py createsuperuser
...`
```

6. Start the development server:

```
``bash
python manage.py runserver 0.0.0.0:8000
...`
```

The API is now accessible at ``http://127.0.0.1:8000/api/``

2. Starting the Frontend (React Native Expo)

1. Open a **new** terminal window.

2. Navigate to the frontend directory:

```
``bash
```

```
cd frontend
```

```
...
```

3. Install the node modules (only needed the first time):

```
```bash
```

```
npm install
```

```
...
```

4. **\*\*IMPORTANT: API URL Configuration\*\***

- If you are testing on an **\*\*Android Emulator\*\***, the API URL in `frontend/src/services/api.js` should be `http://10.0.2.2:8000/api`. (This is already set by default).

- If you are testing on a **\*\*physical device\*\*** or **\*\*iOS Simulator\*\***, change the `API\_URL` in `frontend/src/services/api.js` to your computer's local Wi-Fi IPv4 address (e.g., `http://192.168.1.X:8000/api`).

5. Start the Expo server:

```
```bash
```

```
npx expo start
```

```
...
```

6. ****To view the app:****

- Press ****`a`**** in the terminal to open it in an Android Emulator.

- Press ****`i`**** to open in an iOS Simulator.

- Or download the ****Expo Go**** app on your physical smartphone and scan the QR code displayed in the terminal.

📱 How to Use the App

As a Customer:

1. Open the app and click ****Sign Up****. Register an account.

2. Log in using your new credentials.

3. You will be directed to the ****Customer Home Screen****, where you can browse available services (Deep Cleaning, Junk Removal, etc.).

4. Click ****Add**** on services you need.

5. Navigate to the ****Cart tab**** (bottom center). Review your order and click ****Confirm Booking****.

6. Navigate to the **Profile tab** to view your active and past bookings.

As a Worker:

Note: Role selection is currently automated as 'CUSTOMER' on the basic register screen. To test the worker flow, you can create a worker user via the Django Admin panel at `http://127.0.0.1:8000/admin/` or manually change a user's role in the database.

1. Log in with a `WORKER` account.
2. The app will automatically route you to the **Worker Job Board**.
3. Here you will see all `PENDING` jobs in the system.
4. Click **Accept Job** on any available task.
5. The job status will update to `ACCEPTED`, assigning you to the task.

🏠 Project Structure

```text

HomeAssist/

|

| └─ backend/       # Django Backend

| | └─ backend/       # Core settings, urls, ASGI/WSGI

| | └─ users/        # Custom User Model & Auth APIs

| | └─ services/     # Service & Category Models & APIs

| | └─ bookings/     # Booking & Cart Models & APIs

| | └─ manage.py

|

| └─ frontend/       # React Native Frontend

| | └─ App.js        # App Entry Point

| | └─ src/

| | | └─ navigation/   # AppNavigator (Role-based routing)

| | | └─ screens/     # UI Screens (Auth, Customer, Worker)

| | | └─ services/    # Axios API configuration

| | | └─ store/       # Zustand State Management

| | └─ package.json

```
|
└─ venv/ # Python Virtual Environment
'''
```